Smart Parking System

An IoT-Based Parking Space Occupancy Monitor

Project Report: Advanced Embedded Systems Lab

Mir Sharjil Hasan, Md Sayeed Anwar Seam, and Saracini Kleivi
Hamm-Lippstadt University of Applied Sciences (HSHL)

2026

***Abstract*—We built a small IoT system that tells you whether a parking spot is taken. An HC-SR04 ultrasonic sensor measures the distance to whatever is parked above it, an ESP32 decides between `FREE` and `OCCUPIED`, switches a green or red LED, and publishes the result once per second over Wi-Fi to an MQTT broker running on a Raspberry Pi 3. The Pi stores the data and shows it on a live web dashboard. This report walks through the requirements, the hardware, the wiring, the firmware, and what happened when we tested it.**

I. Introduction

Anyone who has circled a full parking lot knows the problem: you cannot see from the entrance which spots are free. Our idea was to fix that for a single spot first, with hardware cheap enough that scaling up to a whole lot would just mean adding more nodes.

The concept is simple. An ultrasonic sensor sits above the parking spot and constantly measures the distance downwards. If something large is closer than 10 cm to the sensor, a car is there. An ESP32 reads the sensor, lights up a red or green LED on site, and pushes the status over Wi-Fi to a central Raspberry Pi using MQTT a lightweight publish/subscribe protocol that is pretty much the standard for this kind of IoT messaging [1]. The Pi runs the Mosquitto broker [2], logs everything into SQLite, and serves a small Flask dashboard so you can check the spot from your phone.

II. System Overview and Architecture

Figure 1 shows how the pieces fit together. There are three layers: sensing and indication (HC-SR04 and the LEDs), edge processing and communication (ESP32), and the backend (Raspberry Pi with broker, database and dashboard).

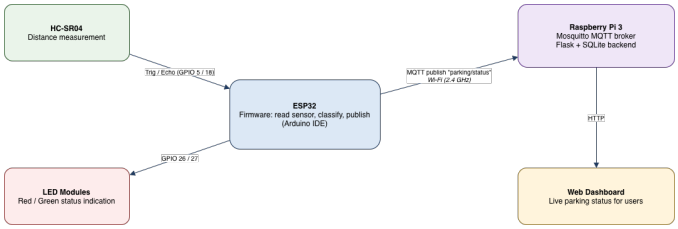


Fig. 1. System architecture. The ESP32 publishes to the MQTT topic `parking/status`; the Raspberry Pi subscribes, stores, and displays.

We deliberately kept all the decision logic on the ESP32. The node works even if the backend is down the LEDs still show the correct status locally and the only thing that travels over the network is a short status string.

III. Requirements

Before wiring anything we wrote down what the system actually has to do. Table I lists the requirements, and Figure 2 shows them as a SysML requirement diagram, including which hardware block satisfies which requirement.

TABLE I
System Requirements.

ID	Title	Description
R1	Smart Parking System	The system shall detect and report the occupancy of a parking space in real time.
R1.1	Occupancy Detection	The system shall measure the distance with an ultrasonic sensor and classify the spot as OCCUPIED if the distance is below 10 cm.
R1.2	Local Indication	The system shall indicate the status via LEDs: green = FREE, red = OCCUPIED.
R1.3	Wireless Reporting	The system shall publish the parking status via MQTT over Wi-Fi to the central broker.
R1.4	Update Rate	The status shall be refreshed and published at least once per second.
R1.5	Robust Connectivity	The system shall automatically reconnect to the MQTT broker after a connection loss.

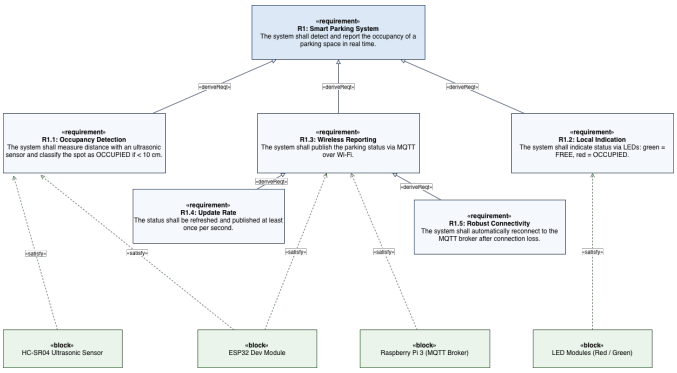


Fig. 2. SysML requirement diagram with «deriveReqt» and «satisfy» relationships.

IV. Hardware Components

Table II is the full bill of materials. Nothing here is exotic – everything came from a standard starter kit plus the Raspberry Pi.

TABLE II
BILL OF MATERIALS.

Component	Qty	Purpose
ESP32 Dev Module	1	Reads the ultrasonic sensor and sends the parking status via Wi-Fi using MQTT.
Raspberry Pi 3 Model B	1	Runs the MQTT broker (Mosquitto) and receives the parking data.
HC-SR04 Ultrasonic Sensor	1	Measures the distance to detect whether the parking space is occupied.
LED Modules (Red & Green)	2	Green = spot free, red = spot occupied.
Breadboard	1	Connects all components without soldering.
Jumper Wires	several	Connect the ESP32, sensor, and LEDs.
MicroSD Card (16 GB) + Adapter	1	Stores Raspberry Pi OS; the adapter is used to flash the OS from a computer.
Micro-USB Cables	2	Power the Raspberry Pi; power and program the ESP32.
HDMI Cable, USB Keyboard & Mouse	1 each	Set up and control the Raspberry Pi.
Wi-Fi Router (Vodafone-2ED3)	1	Puts the ESP32 and the Raspberry Pi on the same network.

Figure 3 shows the four main components. The ESP32 [4] does all the work on the node side: it has built-in Wi-Fi, so no extra radio module is needed. The HC-SR04 [5] is the classic hobbyist distance sensor it fires a 40 kHz ultrasonic burst and reports how long the echo takes to come back. One practical catch: its Echo pin outputs 5 V, while ESP32 GPIOs are only 3.3 V tolerant, so the echo line needs a voltage divider. The Raspberry Pi 3 [6] is comfortably overpowered for the broker job, which is fine it also hosts the database and dashboard. The LED modules [7] plug straight into the breadboard and can be driven directly from a GPIO pin.

(a) ESP32 Dev Module [4].

(b) HC-SR04 ultrasonic sensor [5].

(c) Raspberry Pi 3 Model B [6].

(d) LED status modules [7].

Fig. 3. Main hardware components.

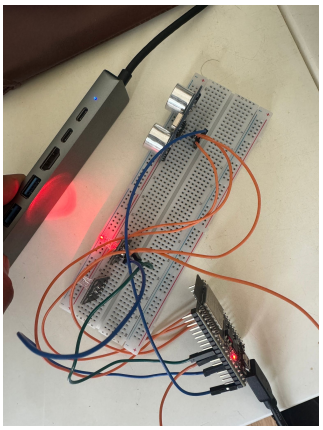
V. CIRCUIT AND WIRING

Table III shows how everything is connected. The trigger and echo lines of the HC-SR04 go to GPIO 5 and GPIO 18, the LEDs sit on GPIO 26 (green) and GPIO 27 (red). As mentioned above, the echo signal passes through a 1 k■/2 k■ voltage divider so the 5 V output does not damage the 3.3 V input of the ESP32.

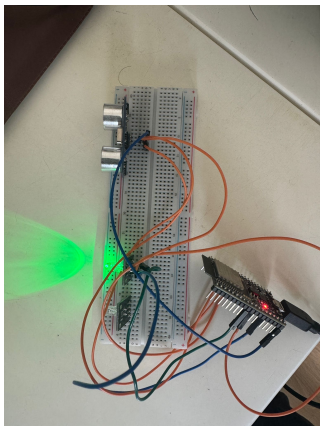
TABLE III
PIN MAPPING BETWEEN THE ESP32 AND THE PERIPHERALS.

Signal	Peripheral pin	ESP32 pin
Trigger	HC-SR04 Trig	GPIO 5
Echo	HC-SR04 Echo	GPIO 18 (via voltage divider)
Green LED	LED module signal	GPIO 26
Red LED	LED module signal	GPIO 27
Power	VCC	5 V (VIN) / 3V3 as required
Ground	GND	GND

Figure 4 shows the prototype on the breadboard during testing once with the spot “occupied” (red LED) and once free (green LED).



(a) Spot occupied red LED on.



(b) Spot free green LED on.

Fig. 4. The assembled prototype during testing.

VI. SOFTWARE IMPLEMENTATION

We wrote the firmware in the Arduino IDE with the ESP32 board package. Wi-Fi comes from the built-in `WiFi` library; for MQTT we used `PubSubClient` [3]. On the Raspberry Pi side, Mosquitto [2] acts as the broker on port 1883.

The logic is one straightforward loop. Trigger the sensor with a 10 µs pulse, measure how long the echo pin stays high, convert that time into a distance with $d = t \cdot 0.0343/2$ (speed of sound, there and back), compare against the 10 cm threshold, set the LEDs, publish the status, wait one second, repeat. If the MQTT connection drops at any point, the firmware blocks in a reconnect loop that retries every two seconds that is what fulfils R1.5.

Listing 1 shows the complete firmware. We redacted the Wi-Fi password for this report.

```
#include <WiFi.h>
#include <PubSubClient.h>

#define TRIG_PIN 5
#define ECHO_PIN 18

#define GREEN_LED 26
#define RED_LED 27

const char* ssid = "Vodafone-2ED3";
const char* password = "********";        // redacted
const char* mqtt_server = "192.168.0.53";

WiFiClient espClient;
```

```
PubSubClient client(espClient);

void setup_wifi() {
  Serial.print("Connecting to WiFi");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT...");
    if (client.connect("ESP32_Parking")) {
      Serial.println("connected");
    } else {
      Serial.print("failed, retrying...");
      delay(2000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  setup_wifi();
  client.setServer(mqtt_server, 1883);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);
  float distance = duration * 0.0343 / 2;

  String status;

  if (distance < 10) {
    status = "OCCUPIED";
    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);
  } else {
    status = "FREE";
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
  }

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.print(" cm | Status: ");
  Serial.println(status);

  client.publish("parking/status", status.c_str());

  delay(1000);
}
```

Listing 1. ESP32 firmware.

VII. Activity Diagram

Figure 5 shows the same control flow as a UML activity diagram. The two decision points“is the broker still connected?” and “is the distance below 10 cm?” are the only branching in the whole program, which is exactly why the node has been so reliable.

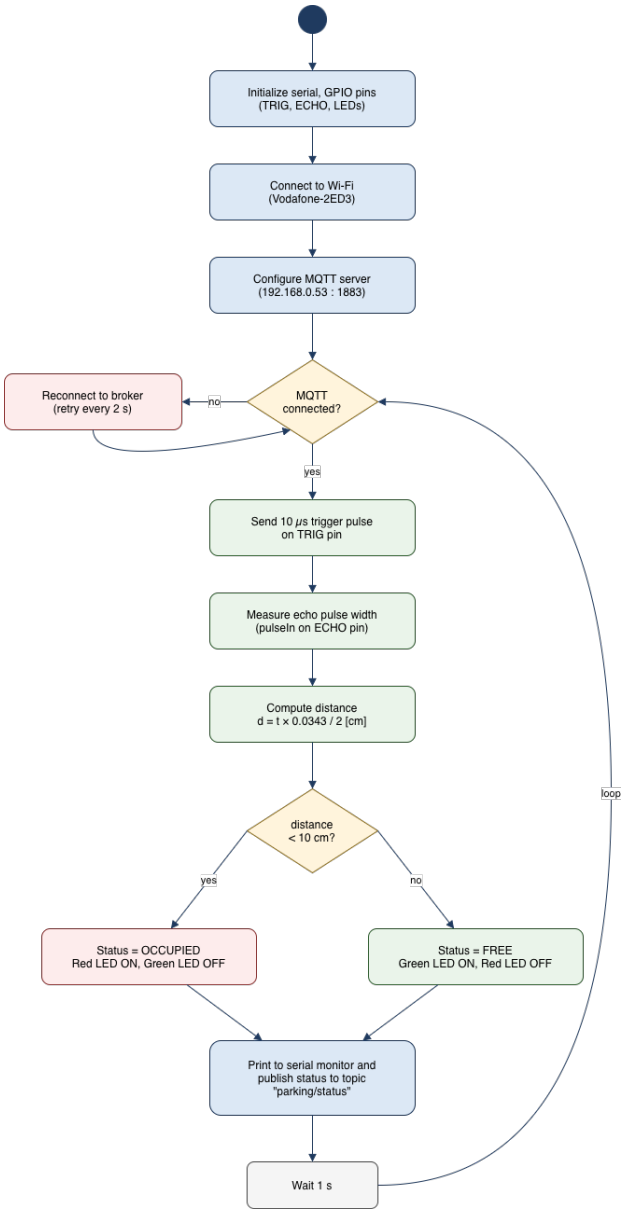


Fig. 5. UML activity diagram of the ESP32 firmware.

VIII. Testing and Results

We tested the prototype by holding an object (standing in for a car) at different distances above the sensor. The behaviour matched the requirements:

- Anything closer than 10 cm flipped the status to `OCCUPIED` red LED on, message published (R1.1, R1.2, R1.3).
- Removing the object flipped it back to `FREE` within a second (R1.4).
- The serial monitor showed stable distance readings, and every status change arrived at the Pi on `parking/status`.

- When we restarted the broker on purpose, the ESP32 reconnected on its own after a couple of retries and carried on publishing (R1.5).

IX. Conclusion

The system does what we set out to build: a cheap, self-contained parking node that reports its status in real time, plus a central backend that collects and displays the data. Every requirement from Table I is met by the prototype.

There is obvious room to grow. More sensor nodes would turn this into a real parking-lot system; the 10 cm threshold could be calibrated per mounting height instead of hard-coded; MQTT should run over TLS before this leaves the lab; and battery operation with deep-sleep phases between measurements would make the node truly wireless.

Work Distribution

Mir Sharjil Hasan: Software implementation, documentation, and diagram work.

Saracini Kleivi: Diagram work and ultrasonic sensor setup.

Md Sayeed Anwar Seam: Hardware implementation.

References

[1] OASIS, "MQTT Version 3.1.1 Specification," 2014. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[2] Eclipse Foundation, "Eclipse Mosquitto: An open source MQTT broker." https://mosquitto.org/

[3] N. O'Leary, "PubSubClient: A client library for MQTT messaging." https://pubsubclient.knolleary.net/

[4] Voltaat, "DOIT ESP32 DevKit V1 Development Board" [product photo]. https://voltaat.com/products/doit-esp32-devkit-v1-development-board

[5] ReversePCB, "HC-SR04 Ultrasonic Sensor Modules" [product photo]. https://www.reversepcb.com/hc-sr04-ultrasonic-sensor/

[6] Raspberry Pi Ltd, "Raspberry Pi 3 Model B" [product photo]. https://www.raspberrypi.com/products/raspberry-pi-3-model-b/

[7] IoTguider, "RGB 3 Color LED Module KY-016" [product photo]. https://iotguider.in/arduino/rgb-3-color-led-module-ky-016-in-arduino/